biomimetics

MDPI

Article

Trajectory Tracking Control for Robotic Manipulator Based on Soft Actor–Critic and Generative Adversarial Imitation Learning

Jintao Hu, Fujie Wang *, Xing Li, Yi Qin, Fang Guo and Ming Jiang

School of Computer Science and Technology, Dongguan University of Technology, Dongguan 523808, China; 221115202@dgut.edu.cn (J.H.); lixing8245@163.com (X.L.); qinyidee@163.com (Y.Q.); 2019091@dgut.edu.cn (F.G.); jiangm@dgut.edu.cn (M.J.)

* Correspondence: wangfujie128@gmail.com

Abstract: In this paper, a deep reinforcement learning (DRL) approach based on generative adversarial imitation learning (GAIL) and long short-term memory (LSTM) is proposed to resolve tracking control problems for robotic manipulators with saturation constraints and random disturbances, without learning the dynamic and kinematic model of the manipulator. Specifically, it limits the torque and joint angle to a certain range. Firstly, in order to cope with the instability problem during training and obtain a stability policy, soft actor–critic (SAC) and LSTM are combined. The changing trends of joint position over time are more comprehensively captured and understood by employing an LSTM architecture designed for robotic manipulator systems, thereby reducing instability during the training of robotic manipulators for tracking control tasks. Secondly, the obtained policy by SAC-LSTM is used as expert data for GAIL to learn a better control policy. This SAC-LSTM-GAIL (SL-GAIL) algorithm does not need to spend time exploring unknown environments and directly learns the control strategy from stable expert data. Finally, it is demonstrated by the simulation results that the end effector of the robot tracking task is effectively accomplished by the proposed SL-GAIL algorithm, and more superior stability is exhibited in a test environment with interference compared with other algorithms.

Keywords: generative adversarial imitation learning; end effector; robotics tracking control; long short-term memory; deep reinforcement learning

check for updates

Citation: Hu, J.; Wang, F.; Li, X.; Qin, Y.; Guo, F.; Jiang, M. Trajectory Tracking Control for Robotic Manipulator Based on Soft Actor–Critic and Generative Adversarial Imitation Learning. *Biomimetics* **2024**, *9*, 779. https://doi.org/10.3390/biomimetics9120779

Academic Editors: Junfei Li and Simon X. Yang

Received: 10 November 2024
Revised: 15 December 2024
Accepted: 18 December 2024
Published: 21 December 2024

Copyright: © 2024 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

With artificial intelligence technology advancing and the concept of Industry 4.0 emerging, the development of robot manipulators has gained significant attention due to their numerous applications in various sectors [1]. The operational scenarios are no longer fixed, predefined, or well known [2]. Regarding trajectory tracking control for a end effector of a manipulator, it is essential for each position to track a desired trajectory closely [3]. Various methods are proposed to achieve satisfactory tracking control effects, including PID control, sliding-mode control, adaptive tracking control, and model predictive control [4–8]. However, many of the mentioned methods necessitate a precise model of the robot, and complex controllers often need more control parameters. Fortunately, neural networks demonstrate exceptional approximation capabilities for uncertain mathematical models and are among the effective methods for addressing nonlinear system control problems.

In [9], an adaptive asymptotic prescribed performance approach using a radial basis function neural network (RBFNN) is proposed for a hydraulic manipulator. An end-to-end (E2E) deep learning method is proposed for robot classification and real-time motion control in [10]. Through forward synchronous learning and adjustable Rectified Linear Unit (ReLU), the stability and robustness of the deep learning algorithm are ensured. However, neural network training for tracking control tasks in robotic manipulators is susceptible to local optimal problems. It is worth mentioning that the appearance of DRL expands the

options for designing control algorithms [11]. The DRL collects experience data through interactive learning of the agent with the environment, utilizing these data to optimize the loss function and train the model. Consequently, local optimal solution problems can be flexibly handled.

In [12], the fuzzy Q-learning network, a typical DRL scheme, is used to resolve the problem of trajectory tracking of an Uncertain Quadrotor System (UQS). The state–action–reward–state–action (SARSA) algorithm is used to realize the positioning of random target and fixed target points of the 3-DoF manipulator end effector in [13]. While Q-learning and SARSA are effective for addressing typical reinforcement learning (RL) tasks, they are not suitable for continuous spaces. It should be noted that trajectory tracking control problems typically involve continuous state spaces. Thus, in order to complete the tracking control task of pneumatic musculoskeletal robots, a model-based RL (MBRL) method is proposed in [14]. In [15], a model-based offline RL method is introduced to enhance the control performance, which is combined with a torque controller to implement tracking control for a manipulator. However, learning an effective control policy is more challenging with model-based RL (MBRL) compared with model-free RL (MFRL). The MFRL algorithms are not dependent on specific environment models, which enables them to perform control tasks effectively in dynamic and uncertain environments. In [16], a distributed Proximal Policy Optimization (PPO) algorithm [17] based on the LSTM network is proposed to train robotic arms and mobile robots to track a given trajectory. In order to control a 2-DoF manipulator with an unknown deadzone to track a desired trajectory, an actor–critic RL method is adopted in [18]. An end-to-end target tracking method utilizing a DRL approach is proposed to address the complexity of control in free-floating space manipulators (FFSMs) in [19]. The challenge of locating and tracking the eddy center in an uncharted environment using an Underactuated Autonomous Surface and Underwater Vehicle (UASUV) is explored in [20], in which the SAC algorithm and LSTM are combined. While MFRL is extensively studied in tracking control tasks, most existing algorithms necessitate numerous training epochs to derive reasonable control policies.

Fortunately, the proposal of GAIL solves the problem of time-consuming training in many tasks [21]. GAIL learns the optimal policy directly from expert data by combining Inverse RL (IRL) [22] with Generative Adversarial Networks (GANs) [23]. In [24], a GAIL-based DRL method is introduced to train an Autonomous Underwater Vehicle (AUV) to emulate expert paths. Similarly, in [25], a GAIL-driven navigation system for Unmanned Surface Vehicles (USVs) is proposed, which learns a policy to mimic expert trajectories. The above-mentioned references [24,25] use the on-policy algorithms PPO and Trust Region policy optimization (TRPO) [26] as generators, respectively [27]. Although the on-policy algorithm is stable, it is challenging to learn a superior control policy due to the expense of sample efficiency GAIL policy training. When the robot environment model is unknown, the exploration ability of the algorithm is crucial. Meanwhile, off-policy methods exhibit superior sample efficiency, but their training process is unstable [28]. Fortunately, by utilizing the LSTM network, the evolving patterns of joint positions over time are competently captured and comprehended, thereby mitigating instability during the training process of robotic manipulators for tracking control tasks. Therefore, the main focus of this paper is to embed LSTM into the SAC algorithm and utilize the resulting SAC-LSTM algorithm as a generator for GAIL training. This approach aims to address the trajectory tracking control problem for the end effector of manipulator systems under input saturation and random interference. The principal contributions are summarized as follows:

(1) Inspired by [20], the LSTM is introduced to enhance the stability of the end effector of the manipulator when tracking the target trajectory. During trajectory tracking control, the sequential states of the manipulator are inputted into the LSTM layer to produce hidden states. These hidden states are processed to generate Gaussian action parameters. This method effectively captures sequential dependencies, thereby improving the robustness and adaptability of trajectory tracking control.

(2) In the above-mentioned references [24,25], the on-policy algorithm is selected as the generator of GAIL. In this paper, the off-policy algorithm SAC combining LSTM is chosen as a generator, and the policies trained by SAC-LSTM are selected as the expert data. Through GAIL training, the agent learns control strategies directly from expert data and is able to quickly learn strategies that are superior to expert data.

(3) A SL-GAIL algorithm is proposed by combining the SAC-LSTM and GAIL method. This algorithm is used to train the robot to track the target trajectory. This approach reduces unnecessary exploration, accelerates the acquisition of superior control strategies, and helps improve the efficiency and robustness of trajectory tracking for the end effector of the robotic manipulator.

The controlled objects and control methods used in the above literature are shown in Table 1.

Table 1. The controlled objects and their control methods.

Category	Method	Main Constraints	References
Hydraulic Manipulators	RBFNN-based adaptive asymptotic method	Complex mathematical models, uncertain dynamics	[9]
Serial Manipulators	PID, sliding-mode control, adaptive control	High reliance on precise models, many control parameters	[4,5,7]
Free-floating Manipulators	DRL-based SAC-RNN	Unknown dynamics, high control complexity	[19]
3-DoF Manipulators	SARSA algorithm	Weak capability for handling continuous state spaces	[13]
2-DoF Manipulators	Actor–critic RL	Deadzone problem, nonlinear dynamic characteristics	[18]
Pneumatic Musculoskeletal Robots	Model-based RL (MBRL) methods	Difficulty in learning effective control policies	[14]
Autonomous Underwater Vehicles (AUVs)	GAIL-based policy learning	Low data efficiency, challenging generalization to unknown environments	[24]
Unmanned Surface Vehicles (USVs)	GAIL combined with PPO or TRPO	Low sample efficiency, insufficient exploration capability	[25]
Robotic Arms and Mobile Robots	Distributed PPO combined with LSTM	Insufficient handling of temporal state dependencies	[16]
Underactuated Autonomous Surface/Underwater Vehicles (UASUVs)	SAC combined with LSTM	Training instability, input disturbances	[20]

2. System Description

2.1. Dynamics Model

The dynamical equations of the system can be obtained by deriving the equations from the Lagrangian functions. The dynamic equation for a n-DOF manipulator is described as follows [29]:

$$M(\theta)\ddot{\theta} + C(\theta,\dot{\theta})\dot{\theta} + G(\theta) = \tau + \tau_d \tag{1}$$

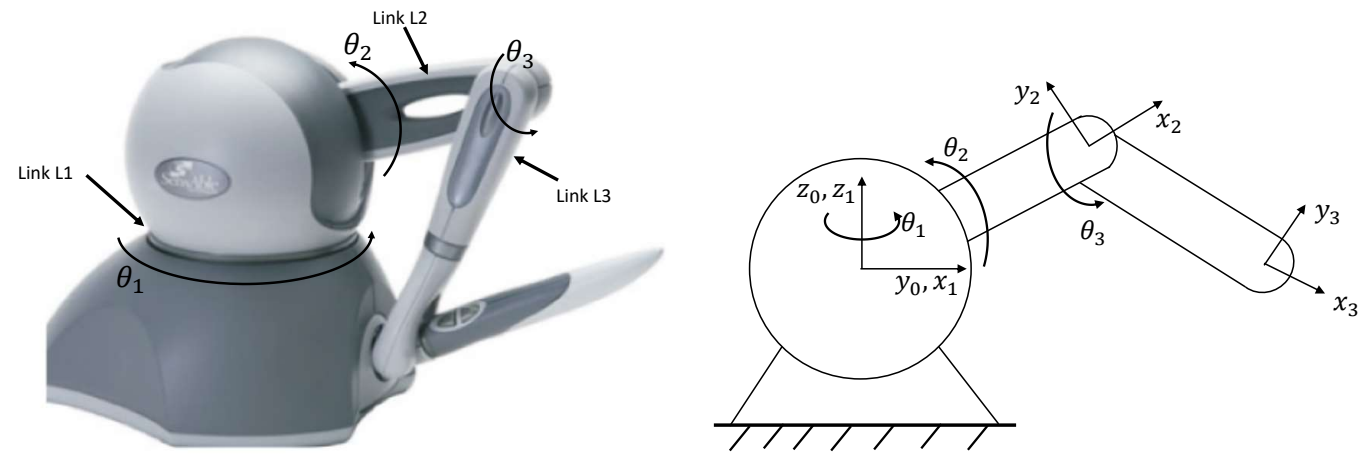
Among them, θ, $\dot{\theta}$, and $\ddot{\theta} \in R^n$ represent the joint position, joint velocity, and joint acceleration, respectively. $M(\theta) \in R^{n\times n}$ expresses the inertia matrix, $C(\theta,\dot{\theta}) \in R^{n\times n}$ represents the centripetal and Coriolis torque, $G(\theta) \in R^n$ is the gravitational force, $\tau \in R^n$ is the to the joint torques, and $\tau_d \in R^n$ is a random interference. In this paper, the Phantom Omni robot is studied. The robot is shown in Figure 1a. The schematic diagram of the Phantom Omni is shown in Figure 1b, in which the reference frames used in dynamics are

outlined [30]. The robot is a 3-DoF manipulator and is modeled by Equation (1), and the M, C, and G matrices are as follow [31]:

$$M(\theta) = \begin{bmatrix} m_{11} & 0 & 0 \\ 0 & m_{22} & m_{23} \\ 0 & m_{32} & m_{33} \end{bmatrix}, G(\theta) = \begin{bmatrix} 0 \\ gk_5c_2 + gk_6c_{23} \\ gk_6c_{23} \end{bmatrix},$$

$$C(\theta, \dot{\theta}) = \begin{bmatrix} -a_1\dot{\theta}_2 & -a_1\dot{\theta}_1 & -a_2\dot{\theta}_1 \\ a_1\dot{\theta}_1 & -a_3\dot{\theta}_3 & -a_3(\dot{\theta}_2 + \dot{\theta}_3) \\ a_2\dot{\theta}_1 & a_3\dot{\theta}_2 & 0 \end{bmatrix}$$

where $m_{11} = k_1 + k_2c_2^2 + k_3c_{23}^2 + 2k_4c_2c_{23}, m_{22} = k_2 + k_3 + 2k_4c_3, m_{23} = k_3 + k_4c_3$, $m_{32} = m_{23}, m_{33} = k_3, a_1 = k_2c_2s_2 + k_3c_{23}s_{23} + k_4c_{2\times23}, a_2 = k_3c_{23}s_{23} + k_4c_2s_{23}$, $a_3 = k_4s_3, s_i = sin(\theta_i), s_{23} = \sin(\theta_2 + \theta_3), c_i = \cos(\theta_i), c_{23} = \cos(\theta_2 + \theta_3)$, $c_{2\times23} = \cos(2\theta_2 + \theta_3)$.



(**a**) (**b**)

Figure 1. (**a**) The 3-DOF Phantom Omni robot. (**b**) Schematic of the 3-DOF device.

2.2. Kinematics Model

The real-time joint angle and joint velocity of the manipulator are obtained by solving Equation (1). Thus, the relationships between the end effector position vector and joint space vector are expressed as [32]

$$x = f(\theta) \tag{2}$$

$$\dot{x} = J(\theta)\dot{\theta} \tag{3}$$

where $f(\cdot) : R^n \to R^n$ is the mapping between the joint space and task space. $J(\theta) \in R^{n\times n}$ is the Jacobian matrix. According to forward kinematics, the Cartesian coordinates of the end effector are expressed as follows [32]:

$$\boldsymbol{x} = \begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} (l_2\cos(\theta_2) + l_3\cos(\theta_2 + \theta_3))\cos(\theta_1) \\ (l_2\cos(\theta_2) + l_3\cos(\theta_2 + \theta_3))\sin(\theta_1) \\ l_2\sin(\theta_2) + l_3\sin(\theta_2 + \theta_3) \end{bmatrix} \tag{4}$$

the Jacobian matrix, J, is expressed [33]

$$J = \begin{bmatrix} -rs_1 & -zc_1 & -l_3c_1s_{23} \\ rc_1 & -zs_1 & -l_3s_1s_{23} \\ 0 & r & l_3c_{23} \end{bmatrix} \tag{5}$$

where $r = \sqrt{x^2 + y^2}$. According to Equations (3) and (5), the $\dot{\boldsymbol{x}}$ is as follows:

$$\dot{\boldsymbol{x}} = \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{z} \end{bmatrix} = \begin{bmatrix} -rs_1 & -zc_1 & -l_3c_1s_{23} \\ rc_1 & -zs_1 & -l_3s_1s_{23} \\ 0 & r & l_3c_{23} \end{bmatrix} \begin{bmatrix} \dot{\theta}_1 \\ \dot{\theta}_2 \\ \dot{\theta}_3 \end{bmatrix} \tag{6}$$

2.3. Control Objective

The kinematics of a robotic manipulator is generally nonlinear, and the parameters of the manipulator system are not completely known, which results in modeling uncertainties. In addition, this paper addresses the task of controlling the joint angles by applying torques to the robot arm's joints, subsequently enabling the end effector to track the time-varying curve in the task space, which adds to the complexity and difficulty of the control process. In this paper, the control goal is designing a RL trajectory tracking controller that aims to solve the time-varying curve tracking control problem in the task space of a manipulator with saturation constraints on joint angle and torque inputs.

3. Preliminaries

3.1. Reinforcement Learning

An RL problem is typically represented as a Markov Decision Process (MDP). An MDP is defined by a tuple $D = (S, A, P, R)$, where S denotes a set of states that the agent might encounter, A signifies a set of actions that the agent chooses from at any given state, P defines the transition dynamics, specifying the probability of moving from one state, s, to another state, s', given a specific action, and R assigns rewards for actions taken in a state and transitioning to another state.

An RL agent aims to study a policy that maps states to actions based on interactions with its environment. This learning process is illustrated in Figure 2. The agent perceives its current state, and chooses and executes an action based on its policy at each time step. Then the environment transitions to the next state according to the action, and provides a reward. The policy is updated based on the received rewards to maximize the long-term cumulative reward. The main goal is for the agent to train an optimal control policy that maximizes the cumulative discounted reward [11].

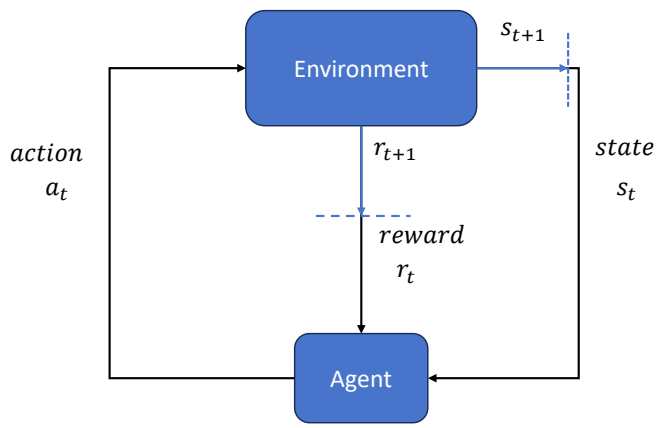


Figure 2. Graph of agent learning with reinforcement learning.

When an RL task conforms to the characteristics of an MDP, the optimization of the task can be approached by resolving the Bellman optimality equations. The optimal state value function, $V^\star(s)$, which maximizes the expected cumulative rewards, satisfies the following Bellman optimality equation [34]:

$$V^\star(s) = \sum_a \pi(s,a) \sum_{s'} P^a_{ss'} \left[R^a_{ss'} + \gamma V^\star(s') \right] \tag{7}$$

The Bellman optimality equation for the state-action value function, $Q^{\star}(s,a)$, is given by:

$$Q^{\star}(s,a) = \sum_{s'} P^{a}_{ss'}\left[R^{a}_{ss'} + \gamma max_{a} Q^{\star}\left(s',a'\right)\right]. \tag{8}$$

By learning the optimal value function, an agent is able to determine the optimal policy by choosing actions that maximize this value function.

3.2. Soft Actor–Critic Algorithm

SAC is the most representative algorithm for actor–critic architecture and has good sampling efficiency. The goal of an ordinary RL algorithm is to maximize the expected return $\sum_{t}^{T} \mathbb{E}_{(s_t,a_t)\sim\pi}[r(s_t,a_t)]$, while SAC is based on the maximum entropy RL theory, which introduces expectations of the policy as the overall optimization goal based on the original RL return [35]:

$$\pi^{\star} = \underset{\pi}{\operatorname{argmax}} \sum t^{T} \mathbb{E}_{(s_t,a_t)\sim\pi}[r(s_t,a_t) + \alpha\mathcal{H}(\pi(\cdot|s_t))] \tag{9}$$

where $\mathcal{H}(\pi(\cdot|s_t)) = -\sum_{a_t} \pi(a_t|s_t)\log\pi(a_t|s_t)$ represents the policy entropy in state s_t, and $\alpha \geq 0$ is a temperature parameter that balances the entropy term against the reward.

SAC utilizes five neural networks: two soft Q-value networks, two target Q-value networks, and a policy network. The soft Q-value network updates follow Equation (8). For a given Q-value network parameterized by θ_i, the loss function is:

$$L_Q(\theta_i) = \mathbb{E}_{(s,a,r,s',d)\sim D}\left[\frac{1}{2}\left(Q_{\theta_i}(s,a) - y(r,s',d)\right)^2\right] \tag{10}$$

where $y(r,s',d)$ is described as:

$$y(r,s',d) = r + \gamma\left(\min_{i=1,2} Q_{\theta'_i}(s',\tilde{a}') - \alpha\log\pi_{\phi}(\tilde{a}'|s')\right) \tag{11}$$

SAC employs the smaller value from the two target Q-value networks to ensure stable training. The Q-value networks are updated using stochastic gradient descent:

$$\nabla_{\theta_i} L(\theta_i) = \nabla_{\theta_i} Q_{\theta_i}(s,a)\left[\left(Q_{\theta_i}(s,a) - y(r,s',d)\right)^2\right] \tag{12}$$

For policy network $\pi_{\phi}(a|s)$, which can be updated directly by maximizing (minimizing negative values), the Q value and policy corresponding to the current policy network:

$$J_{\pi}(\phi) = \mathbb{E}_{(s)\sim D}\left[\mathbb{E}_{a\sim\pi_{\phi}}\left[\alpha\log\pi_{\phi}(a|s) - Q_{\theta_i}(s,a)\right]\right] \tag{13}$$

SAC uses the reparameterization trick to sample actions $a = f_{\phi}(\xi_a;s)$, where $\xi_a \in \mathbb{R}^{\dim(\mathcal{A})}$ is drawn from a fixed Gaussian distribution, and f_{ϕ} represents the reparameterized policy network. The policy network is then updated using stochastic gradient descent:

$$\begin{aligned}\nabla_{\phi} J_{\pi}(\phi) = \nabla_{\phi}\alpha\log\pi_{\phi}(a|s) + (\nabla_{\phi}\alpha\log\pi_{\phi}(a|s) \\ -\nabla Q_{\theta_i}(s,a))\nabla_{\phi} f_{\phi}(\xi_a;s)\end{aligned} \tag{14}$$

3.3. Long Short-Term Memory

Each cell in an LSTM network is composed of three integral components: the forget gate, the input gate, and the output gate. The forget gate controls which information from the previous cell state should be discarded. It is defined as follows: [36]:

$$f_t = \sigma\left(W_f \cdot [h_{t-1}, x_t] + b_f\right) \tag{15}$$

The input gate decides which values are updated and stored in the cell state. This process is broken down into two parts: a candidate value, $\hat{C}_t$, is created using the tanh function; and the input gate, ι_t, controls the extent to which this candidate value is incorporated into the cell state:

$$\begin{aligned} \iota_t &= \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \\ \tilde{C}_t &= \tanh(W_C \cdot [h_{t-1}, x_t] + b_C) \end{aligned} \tag{16}$$

where the final component is the output gate, which determines the output of the LSTM cell. This gate combines the current cell state and the output gate activation to produce the final hidden state:

$$\begin{aligned} o_t &= \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \\ h_t &= o_t \cdot \tanh(C_t) \end{aligned} \tag{17}$$

The mention above σ is sigmoid function, W and b represent the weight matrix and bias term, h_{t-1} is the previous hidden state, x_t is the current input, $[\cdot]$ denotes the concatenation operation, $\hat{C}_t$ is the candidate cell state, and C_t is the cell state. The interaction of these gates enables the LSTM unit to capture long-term dependencies, making it effective for sequential data processing.

3.4. Generative Adversarial Imitation Learning

GAIL enhances traditional imitation learning by incorporating principles from maximum entropy inverse reinforcement learning. It combines imitation learning with Generative Adversarial Networks (GANs) to directly learn the optimal policy that makes the distribution of the state–action pairs of the agent closely match those of expert trajectories. In GAIL, the role of the discriminator is to differentiate between the state–action pairs generated by the agent and those from the expert. Meanwhile, the generator, which represents the policy of the agent, is trained to produce state–action pairs that the discriminator classifies as expert-like. This adversarial process aims to find a saddle point (π, D) in the following objective function [37]:

$$\mathbb{E}_\pi[\log D((s,a))] + \mathbb{E}_{\pi_\mathbb{E}}[\log(1 - D(s,a))] - \lambda H(\pi) \tag{18}$$

where $D(s,a)$ represents the discriminator in GAIL. $\mathbb{E}_\pi[\log D((s,a))]$ is the expectation over the policy of the agent, encouraging the generator to produce state–action pairs that the discriminator classifies as expert-like. $\mathbb{E}_{\pi_\mathbb{E}}[\log(1 - D(s,a))]$ is the expectation over the policy of the expert, ensuring that the discriminator correctly identifies expert state–action pairs. $H(\pi) \triangleq \mathbb{E}_\pi[-\log \pi(a \mid s)]$ denotes the entropy of the policy, π, which encourages exploration and prevents the policy from collapsing to deterministic behaviors. λ is the weighting parameter that determines the significance of the entropy term in the overall objective.

4. Control Design

4.1. Design of System Input/Output

In deep RL, the actor network determines an output action based on the current input state. For the trajectory tracking in the task space of the manipulator, the joint angles of the manipulator and the position of the end effector are crucial. The traditional state space includes joint angles and joint angular velocities, which can describe the internal state of the robot, but cannot directly reflect the relationship between the performance of the end effector of the robot and the target. Therefore, adding the mapped end effector position and velocity, as well as the target position and velocity, can provide more relevant information about the end effector and target trajectory, thereby helping the reinforcement learning algorithm to control more accurately. Furthermore, to guarantee that the control policy is able to obtain effective tracking control, there must be error information for trajectory tracking in the state space [38]. In this paper, the primary objective of the controller is to determine the appropriate actions according to the real-time status information of the

robotic manipulator to achieve effective tracking control. The state space is described as follows:

$$s = \begin{bmatrix} \theta & \dot{\theta} & x & \dot{x} & x_d & \dot{x_d} & e & \dot{e} \end{bmatrix}^T \tag{19}$$

where x and $\dot{x}$ are are the end position and the velocity at the end position, respectively. x_d and $\dot{x_d}$ are the target end position and the velocity at the target end position. $e = x - x_d$ represents the tracking error, and $\dot{e}$ is the the speed error. Additionally, the saturation constraints are imposed on the manipulator joints to constrain the swing amplitude when the manipulators track a target curve. The limitations are as follows:

$$q_i = \begin{cases} q_{\min} & \text{, if } q_i \leq q_{\min} \\ q_i & \text{, if } q_{\min} < q_i < q_{\max}, i = 1, 2, \ldots, n \\ q_{\max} & \text{, if } q_i \geq q_{\max} \end{cases} \tag{20}$$

where $q_{\max} = \pi$ and $q_{\min} = -\pi$.

In trajectory tracking control of the robot manipulator end effector, the controller's output signals are the torque acting on the joint angle of the manipulator. The action space is as follows:

$$a = \begin{bmatrix} \tau \end{bmatrix}^T \tag{21}$$

where

$$\tau_i = \begin{cases} \tau_{\min} & \text{, if } \tau_i \leq \tau_{\min} \\ \tau_i & \text{, if } \tau_{\min} < \tau_i < \tau_{\max}, i = 1, 2, \ldots, n \\ \tau_{\max} & \text{, if } \tau_i \geq \tau_{\max} \end{cases} \tag{22}$$

where $\tau_{\max}$ is the maximum of the input torque, $\tau_{\min}$ is the minimum of the input torque, and $\tau_{\max} = 5$, $\tau_{\min} = -5$.

In RL tasks, the reward is a measure to access the effectiveness of the current policy. In the trajectory tracking task of the manipulator, the tracking error, $e(t)$, is the variable of most concern. Therefore, the reward function is set to a nonlinear form. The reward function is formulated as:

$$r = \left(1 - \frac{1}{\exp(-\varsigma \cdot (\bar{e} - \epsilon))}\right) \tag{23}$$

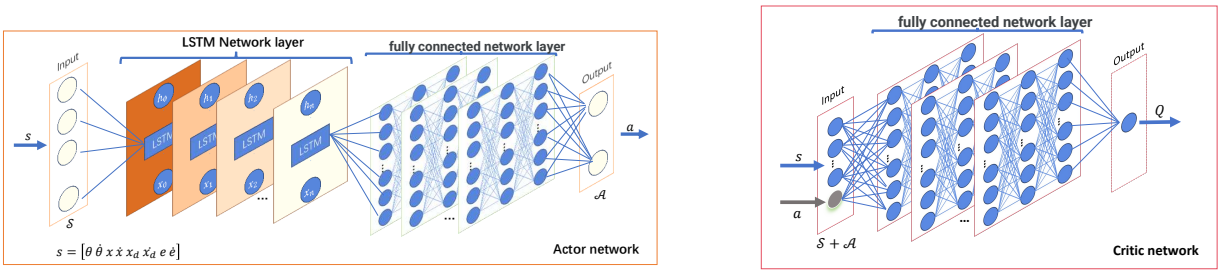
where ς denotes a sensitive scale and is set to adjust the rate of increase, enabling the agent to quickly obtain rewards once a certain performance level is reached. ϵ represents a benefit threshold and is used to indicate the bound of rewards [39]. In this paper, $\varsigma = 2$ and $\epsilon = 0.02$.

4.2. Control Algorithm Based on SAC-LSTM

The actor–critic architecture is employed in most RL algorithms to effectively address policy gradient-based problems. The actor network will eventually train a policy, π, which is the controller required for the tracking task. This policy is refined employing the policy gradient approach under the guidance of the critic network. Concurrently, the critic network needs to be trained to precisely assess the outputs generated by the actor network.

In this paper, the SAC-LSTM employs the actor–critic architecture, including the policy network and critic network, as detailed in Algorithm 1. The network structure is depicted in Figure 3. The actor network includes an LSTM network layer, a fully connected network layer, and an output layer. The state sequence of each time step of the robot manipulator in the trajectory tracking task is used as the input of LSTM, and the hidden state, h, of each time step is used as the output. Then, h at each time step in the entire trajectory is used as the input of the fully connected layer. After passing through the fully connected layer network, the Gaussian distribution parameters of the action (torque) are output. During training, the action values are sampled from this distribution, ensuring that the actions of the agent are chosen in a random and exploratory manner. As training progresses, the network converges and the variance decreases gradually. The value network includes

a fully connected layer and an output layer, taking the splicing of state sequences and actions as input. The network output layer outputs a single variable, the value of the state sequence, $V_\phi(S)$.



(**a**) (**b**)

Figure 3. (**a**) Actor network architecture; (**b**) critic network architecture.

Algorithm 1 SAC-LSTM

Input: Policy network parameter ϕ, Q Network parameters θ_1, θ_2
Input: Empty replay buffer $\mathcal{D}$
Input: target network weights $\hat{\theta}_1 \leftarrow \theta_1, \bar{\theta}_2 \leftarrow \theta_2$
Output: Optimized parameters

```
for each iteration do
    for each interaction with environment do
        Get s and choose a ~ π_φ(. | s_t)
        Perform action a
        Record s', reward r, and
            termination signal d
        Save (s, a, r, s', d) in replay buffer D
        if termination signal d is terminal then
            Reset state of environment
        end if
    end for
    for each gradient do
        From D sample a batch of data B=(s, a, r, s', d);
        Calculate targets for the Q networks following (15)
        Update parameters of Q network according to
            gradient descent following (16)
        Update parameters of policy according to
            gradient descent following (18)
        Update target network parameters with
            φ_targ,i ← ρφ_targ,i + (1 − ρ)φ_i    for i = 1, 2
    end for
end for
```

Remark 1. *Inspired by the above reference [20], the LSTM network is able to efficiently capture sequence information. It generates current outputs by integrating evolving trends from past data, which is more effective for tasks with periodic changes such as robotic manipulator trajectory tracking. In this paper, retaining the outputs of all time steps from the LSTM and passing them to the fully connected layer offers significant advantages for tracking the trajectory of a robotic manipulator. Firstly, this approach fully utilizes the information from the entire sequence, ensuring that the state at each time step is considered, thereby enhancing the precision of trajectory tracking. Secondly, it helps capture long-term dependencies, enabling the model to better handle complex temporal dependencies. Thirdly, using information from the entire sequence can make the control output smoother, reducing jitter and abrupt changes in the robotic arm's movement. Additionally,*

this method improves the model's robustness when dealing with dynamic environments or uncertain external disturbances, enhancing its ability to adapt and respond to changes.

4.3. GAIL for Robot Manipulator Trajectory Tracking

GAIL is able to learn control policies directly from expert demonstration data, which effectively solve the time-consuming problem of reinforcement learning training. The SAC-LSTM algorithm is used as a generator to train GAIL, for implementing SL-GAIL on robotic manipulators' trajectory tracking control, as shown in Figure 4. The discriminator of GAIL is composed of a fully connected layer followed by an output layer. Among them, the state–action pair spliced into the state and action is used as the input of the discriminator, and the output is a value between 0 to 1. The robotic manipulator agent interacts with the environment employing the current policy $\pi(a|s;\theta_n ow)$, which serves as the generator. This interaction generates a trajectory $\Omega = [s_1, a_1, s_2, a_2, \ldots, s_n, a_n]$ consisting of state–action pairs. For each state–action pair (s, a), the discriminator outputs a value, $D(s, a)$, to determine whether it originates from the expert demonstration or from the policy of the agent. Ideally, $D(s, a)$ should be close to 1 for expert trajectories and close to 0 for agent-generated trajectories. The generator (policy of the agent) is trained employing the SAC-LSTM algorithm to produce state–action pairs that closely match the expert data, thereby fooling the discriminator. The discriminator, conversely, is designed to differentiate between the expert and agent-generated trajectories. Eventually, the generator is trained to maximize $\mathbb{E}_\pi[\log(D(s, a))]$ and the discriminator is trained to minimize $\mathbb{E}_\pi[\log(D(s, a))] + \mathbb{E}_{\pi_\mathbb{E}}[\log(1 - D(s, a))]$. GAIL uses a surrogate reward:

$$r(s, a) = -\log(D(s, a)) \tag{24}$$

This surrogate reward is used to update the policy of the agent. The ultimate objective is learning an optimal control policy that maximizes the expected cumulative discounted return:

$$\pi^\star = \arg\min_{\pi \in \Pi} E_\pi[r(s, a)], \tag{25}$$

where $E_\pi[r(s, a)] \triangleq E[\sum_t^T \gamma^t r(s, a)]$ represents an expectation of discounted return according to the trajectory generated by policy π.

This approach, detailed in Algorithm 2, ensures that the policy of the agent evolves to closely mimic expert behavior while efficiently handling the challenges of trajectory tracking control in robotic manipulators.

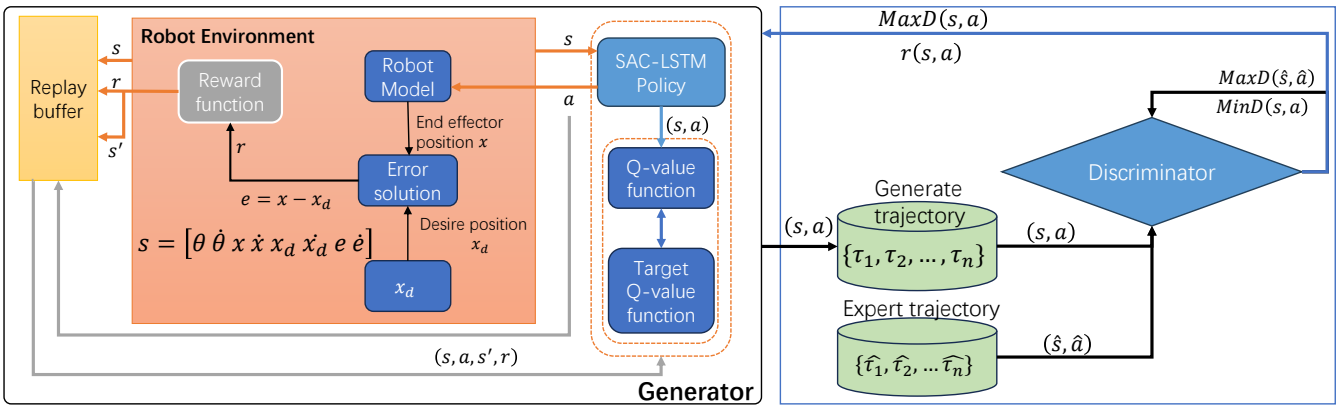


Figure 4. Illustration of the SL-GAIL method.

Remark 2. *Different from the above-mentioned references [24,25], this paper employs the off-policy algorithm SAC as a generator for GAIL. The policy entropy regularization of the SAC algorithm enhances the sampling capability and generates smoother and continuous action outputs, which is*

more suitable for processing systems such as robotic manipulators. In order to solve the instability of the off-policy algorithm when training the GAIL strategy, the LSTM network is added to the SAC algorithm. LSTM effectively captures the long-term dependencies and dynamic patterns in the sequence data, thereby helping the generator to better predict the continuous action sequence, maintain the temporal coherence of the generated samples, and enhance the model's adaptability to dynamic changes in the environment, thereby improving the stability of the generator in the GAIL framework.

Algorithm 2 SL-GAIL

Input: Expert demonstration data $\mathcal{D}_E$, Policy network parameter ϕ, Discriminator network parameters ω
Output: Optimal policy parameters ϕ

```
Initialize policy network parameters φ
Initialize discriminator network parameters ω
for each training iteration do
    Sample expert batch B_E from D_E
    Generate trajectory Ω using current policy π(a|s;φ)
    for each state–action pair (s,a) in Ω do
        Compute discriminator output D(s,a)
        Compute surrogate reward by Equation (24)
    end for
    Update φ using surrogate reward r(s,a)
    Sample state–action pairs (s,a) from Ω and B_E
    Update discriminator parameters ω using Equation (18)
end for
```

5. Simulation

In this section, simulation experiments are performed based on the simulation environment in Section 2 to check the capability of the proposed method in this paper. The SAC, SAC-LSTM, and SAC-GAIL algorithms are chosen as the benchmarks to evaluate our SL-GAIL algorithm. Meanwhile, the anti-interference capabilities of the four controllers are evaluated. In this paper, all experiments are examined using the Phantom Omni robotic manipulator. The initial state values are defined as $\theta_1(0) = -0.3$, $\theta_2(0) = 0.3$, and $\theta_3(0) = -0.8$. The total steps are 3000, and the step size of each step is 0.01 s.

In the all simulation experiments, x_d is selected as

$$\boldsymbol{x_d} = \begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} 0.1\sin(t) + 0.12 \\ 0.1\cos(t) + 0.12 \\ 0.1\sin(t) \end{bmatrix}$$

where $t \in (0, t_{limit})$ and t_{limit} = 30 s.

5.1. Training Performance

In this experiment, the performance of the SAC-LSTM, SAC-GAIL and SL-GAIL algorithms are tested during the training process. The detailed parameter settings of all algorithms are shown in the Appendix A.

The trends in cumulative reward values obtained by the three algorithms, and the standard deviation and variance of the cumulative reward are shown in Figure 5. The results show that the cumulative rewards value of the SL-GAIL agent tend to converge around the 20th episode and the learning curve is smooth later. However, the SAC-LSTM agent and the SAC-GAIL agent do not converge until after the 100th. At the same time, the variance and standard deviation of SL-GAIL are in a stable state after the 100th episode, but the variance and standard deviation of SAC-LSTM and SAC-GAIL always fluctuate greatly.

Moreover, compared with SAC-GAIL, the SAC-LSTM algorithm is more stable during training. Therefore, when using RL to perform robotic manipulator tracking control tasks,

LSTM can enhance the stability of robotic manipulator agent learning and GAIL to improve the efficiency of robotic manipulator agent learning.

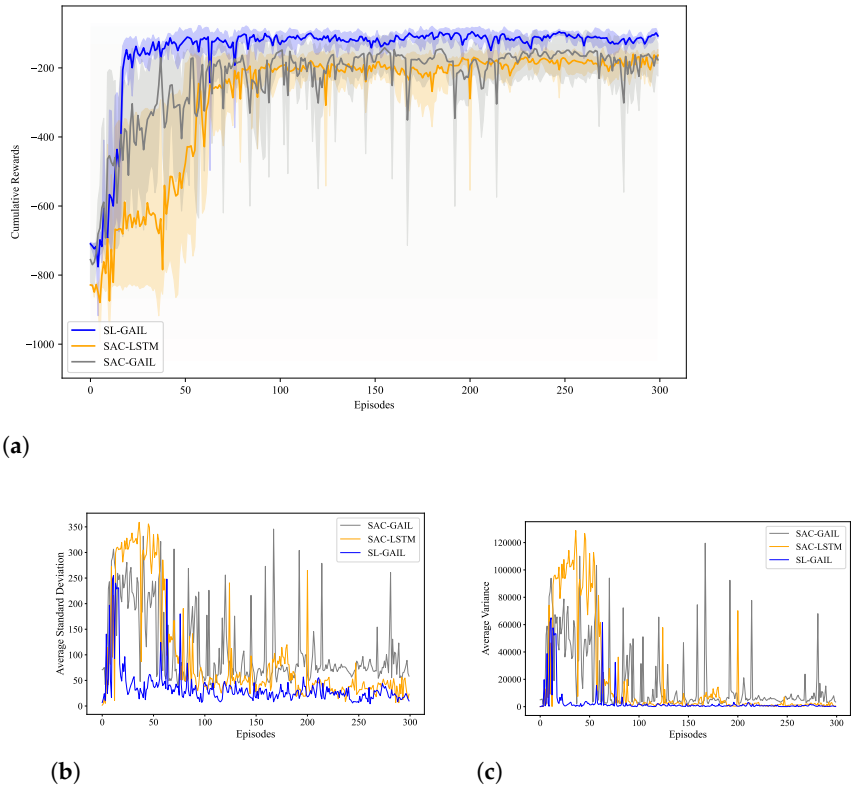


Figure 5. The variance and standard deviation of the cumulative reward value for each episode. (**a**) Cumulative rewards for each episode. (**b**) Standard deviation of cumulative rewards. (**c**) Variance deviation of cumulative rewards

5.2. Control Performance

The tracking control performance of SL-GAIL is compared with SAC, SAC-LSTM, and SAC-GAIL. Figure 6 shows the tracking trajectories, tracking error, and a change in the torque of the end effector using SAC, SAC-GAIL, SAC-LSTM, and SL-GAIL during the training process in the control task. As demonstrated in Figure 6a–c, the trajectories of SAC, SAC-GAIL, SAC-LSTM, and SL-GAIL agents in the X-, Y-, and Z-axes are shown. The tracking errors of the end effector in the three axes are shown in Figure 6d–f. It can be seen that the SL-GAIL algorithm achieves more superior tracking control performance than the baseline methods. Figure 6g–i show the change trend of the torque for the joint; it can be seen that the torque of the robotic manipulator when controlled by SL-GAIL is smaller than that of SAC, SAC-GAIL, and SAC-LSTM.

As can be seen from this experiment, SL-GAIL combines the strengths of GAIL and LSTM and quickly learns policies that surpass those derived from stable expert data. As a result, it achieves excellent control performance with lower torque requirements. The average value of the absolute value of the tracking error (AAE), the average value of the absolute value of torque (AAT) and root-mean-squared error (RMSE) are shown in Table 2. The following formulas calculate AAE, AAT, and RMSE:

$$\mathrm{RMSE} = \sqrt{\frac{1}{T}\sum_{i=1}^{T} e_i^2} \quad i = 1, 2, \ldots, n \tag{26}$$

$$\mathrm{AAE} = \frac{1}{T}\sum_{i=1}^{T} |e_i| \quad i = 1, 2, \ldots, n \tag{27}$$

$$\text{AAT} = \frac{1}{T}\sum_{i=1}^{T}|\tau_i| \quad i = 1, 2, \ldots, n \tag{28}$$

where T is the total time.

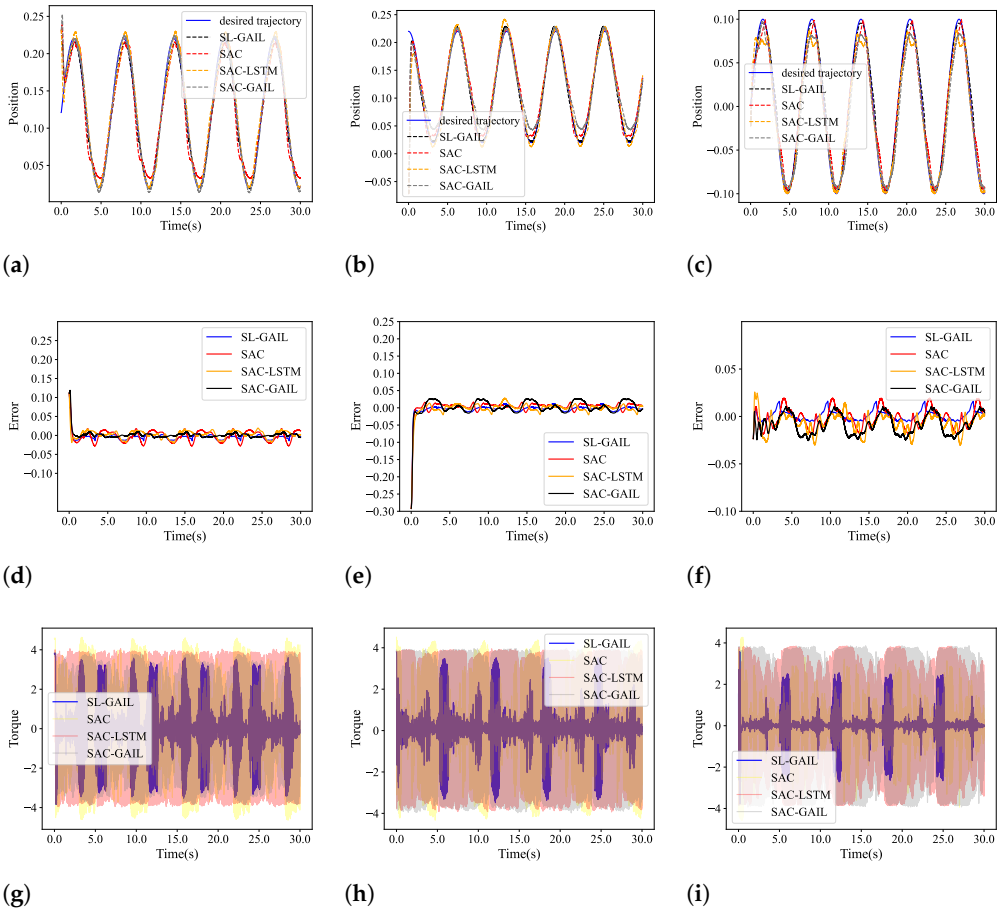


Figure 6. The tracking position, tracking error, and torque in Experiment 1. (**a**–**c**) Tracking position of x-, y-, and z-axes. (**d**–**f**) Tracking error of of x-, y- and z-axes. (**g**–**i**) Torque of the three joints.

Table 2. AAE, AAT, and RMSE of SL-GAIL, SAC-LSTM, and SAC-GAIL algorithms in training.

Axe	Algorithms	SAC	SAC-GAIL	SAC-LSTM	SL-GAIL
X	AAE	0.0123	0.0044	0.0062	0.0058
	AAT	3.5349	1.9642	2.2820	1.1057
	RMSE	0.0158	0.0112	0.0110	0.0101
Y	AAE	0.0088	0.0123	0.0072	0.0066
	AAT	3.4609	2.6186	1.8884	0.7469
	RMSE	0.0206	0.0240	0.0213	0.01975
Z	AAE	0.0081	0.0081	0.0063	0.0038
	AAT	2.9630	2.3313	0.9714	0.4388
	RMSE	0.0099	0.0088	0.0116	0.0052

5.3. Anti-Interference Performance

In this simulation experiment, the proposed method is tested for anti-interfere, y_d is the same as in Experiment 1, and the external disturbance, τ_d, is selected as:

$$\tau_d = \begin{cases} \begin{pmatrix} 5\sin(t) & 5\cos(t) & 5\sin(t) \end{pmatrix}^T & a > 0.8 \\ 0 & \text{else} \end{cases}$$

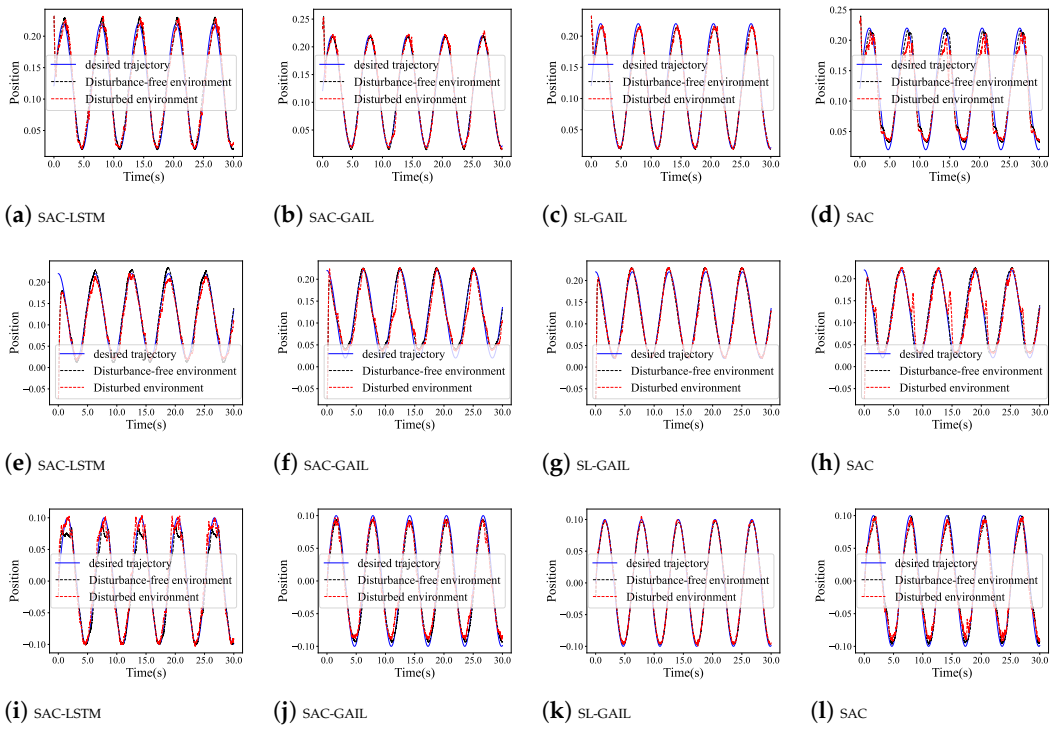
where a represents a random variable uniformly distributed in the interval [0.0, 1.0). If this value exceeds 0.80, a nonlinear disturbance term is added to the system input signal, which means that the perturbation has a twenty percent probability of occurring at each time step. The nonlinear disturbance term is a 3-dimensional column vector composed of the sine and cosine values of the current time step, multiplied by a scaling factor (5). By introducing random disturbances, the robustness of the controller in uncertain environments can be tested. In addition, introducing perturbations in the form of sine and cosine curves can simulate many nonlinear effects in mechanical systems and improve the accuracy of the model.

In order to evaluate the anti-interfere of SL-GAIL, we saved the optimal control policies trained with SAC-LSTM, SAC-GAIL, and SL-GAIL in the curve-following task. The control policies of the three agents are compared with the SL-GAIL algorithm. Figure 7 shows the position tracking of the four algorithms in a disturbance-free environment (DFENV) and a disturbed environment (DENV). The first, second, and third rows are the X, Y, and Z positions of the four algorithms, respectively. It is noted that, compared with the other three algorithms, the position tracking of the SL-GAIL algorithm changes very little in the two environments. The tracking error and torque changes of the four algorithms in the DENV are shown in Figure 8. It can be seen that the torques using SL-GAIL are still smaller than using SAC, SAC-LSTM, and SAC-GAIL. Figure 9 present the AAE and RMSE for all algorithms in the DFENV and DENV. The AAE and RMSE of SL-GAIL tested in the two environments have almost no change. However, the AEE and RMSE of the SAC, SAC-LSTM, and SAC-GAIL change significantly in the two environments. The changes in AAE under the two environments are shown in Table 3. The changes in RMSE under the two environments are shown in Table 4.

From this experiment, it can be seen that, in the robotic arm environment with the same joint angle constraints and maximum torque limits, the anti-interference ability of the SL-GAIL controller proposed in this paper is stronger than that of the three other controllers. It is indicated again that GAIL and LSTM play an important role in this paper in enhancing the robustness of the algorithm.

Table 3. Comparison of AAE in two environments.

Axes	Algorithms	SAC	SAC-GAIL	SAC-LSTM	SL-GAIL
X	DFENV	0.0123	0.0044	0.0062	0.0058
	DENV	0.0176	0.0061	0.0098	0.0059
Y	DFENV	0.0088	0.0123	0.0072	0.0066
	DENV	0.0137	0.0177	0.0118	0.0066
Z	DFENV	0.0081	0.0081	0.0063	0.0038
	DENV	0.0099	0.0102	0.0098	0.0042

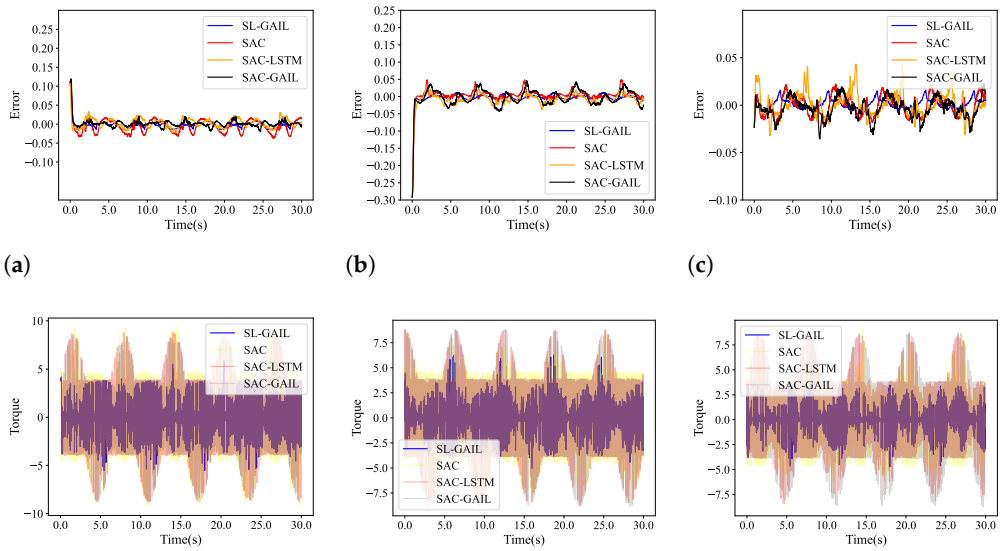


(**a**) SAC-LSTM (**b**) SAC-GAIL (**c**) SL-GAIL (**d**) SAC

(**e**) SAC-LSTM (**f**) SAC-GAIL (**g**) SL-GAIL (**h**) SAC

(**i**) SAC-LSTM (**j**) SAC-GAIL (**k**) SL-GAIL (**l**) SAC

Figure 7. The position tracking in an environment with interference in Experiment 2. (**a**–**d**) Position tracking of x-axes. (**e**–**h**) Position tracking of y-axes. (**i**–**l**) Position tracking of z-axes.



(**a**) (**b**) (**c**)

(**d**) (**e**) (**f**)

Figure 8. The tracking error and torque in an environment with interference in Experiment 2. (**a**–**c**) tracking error of x-, y- and z-axes. (**d**–**f**) tracking error of the three joints.

Table 4. Comparison of RMSE in two environments.

Axes	Algorithms	SAC	SAC-GAIL	SAC-LSTM	SL-GAIL
X	DFENV	0.0158	0.0112	0.0110	0.0101
	DENV	0.0221	0.0123	0.0124	0.0102
Y	DFENV	0.0206	0.0240	0.0213	0.0197
	DENV	0.0274	0.0279	0.0234	0.0196
Z	DFENV	0.0099	0.0088	0.0116	0.0049
	DENV	0.0128	0.0124	0.0127	0.0052

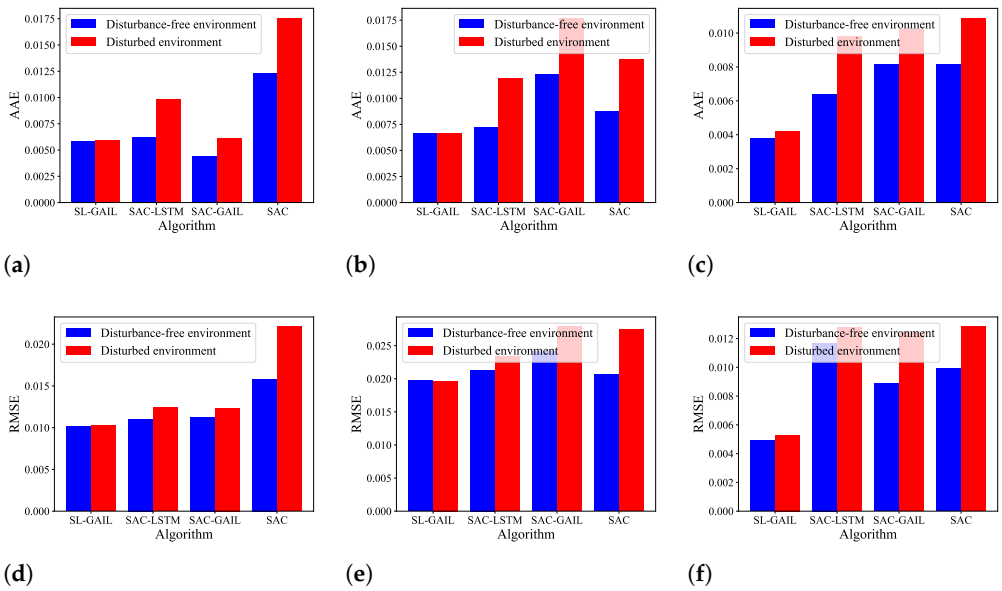


Figure 9. The AAE and RMSE in two environments. (**a**–**c**) The AAE in two environments. (**d**–**f**) The RMSE in two environments.

6. Conclusions

In this paper, we addressed the challenge of trajectory tracking in task space for the end effector of the Phantom Omni manipulator. We proposed the SAC-LSTM algorithm to enhance the performance of the control system, particularly in adapting to time-varying trajectories. The integration of long short-term memory (LSTM) allowed the system to effectively capture temporal dependencies in the trajectory, thereby improving the robustness and adaptability of the controller in dynamic environments. By combining SAC-LSTM with generative adversarial imitation learning (GAIL), the SL-GAIL method is able to directly learn from expert demonstrations, significantly improving the efficiency of policy learning and reducing training time. The simulation results demonstrate that the SL-GAIL method not only achieves faster learning but also enhances the robustness of the control system compared with the baseline algorithm. The experimental outcomes highlight the effectiveness of combining reinforcement learning with imitation learning to solve real-world robotic control problems, suggesting that the proposed approach is a promising solution for time-sensitive and high-precision tasks in robotic manipulation.

Author Contributions: Conceptualization, J.H. and F.W.; Methodology, F.W.; Software, J.H.; Validation, J.H.; Formal analysis, Y.Q.; Resources, X.L. and Y.Q.; Writing original draft, J.H.; Writing review and editing, J.H. and F.W.; Project administration, F.G. and M.J.; Funding acquisition, F.W. and X.L. All authors have read and agreed to the published version of the manuscript.

Funding: This research was supported in part by National Natural Science Foundation of China under grant no. 62203116, 62103106, 62273095, in part by GuangDong Basic and Applied Basic Re-

search Foundation 2024A1515010222, in part by Characteristic Innovation Foundation of Guangdong Education Department under grant 2022ktscx138, 2022ZDZX1031, in part by Liaoning Natural Science Foundation (2022-KF-21-06), in part by Dongguan Science and Technology of Social Development Program under grant no. 20231800935882, SSL Sci-tech Commissioner Program (20234430-01KCJ-G, 20234371-01KCJ-G).

Institutional Review Board Statement: Not applicable.

Data Availability Statement: Data are contained within the article.

Acknowledgments: This research was supported in part by National Natural Science Foundation of China under grant no. 62203116, 62103106, 62273095, in part by GuangDong Basic and Applied Basic Research Foundation 2024A1515010222, in part by Characteristic Innovation Foundation of Guangdong Education Department under grant 2022ktscx138, 2022ZDZX1031, in part by Liaoning Natural Science Foundation (2022-KF-21-06), in part by Dongguan Science and Technology of Social Development Program under grant no. 20231800935882, SSL Sci-tech Commissioner Program (20234430-01KCJ-G, 20234371-01KCJ-G).

Conflicts of Interest: The authors declare no conflicts of interest.

Appendix A

Table A1. Parameter values of manipulator.

Parameter	Value	Parameter	Value
g	9.806 m/s^2	l_1	0 m
l_2	0.135 m	l_3	0.13 m
k_1	3.7×10^{-3} kg $\cdot$ m^2	k_2	7.0×10^{-3} kg $\cdot$ m^2
k_3	8.0×10^{-3} kg $\cdot$ m^2	k_4	0.4×10^{-3} kg $\cdot$ m^2
k_5	9.1×10^{-3} kg $\cdot$ m^2	k_6	5.2×10^{-3} kg $\cdot$ m^2

Table A2. Parameter values of algorithms.

Parameter	Value	Parameter	Value
learning rate of actor	0.0003	learning rate of critic	0.003
learning rate of discriminator	0.001	learning rate of α	0.0003
soft update parameter	0.005	buffer size	1,000,000
minimal size	5000	batch size	256
lstm hidden size	128	lstm num layers	1

Table A3. Symbol descriptions for reinforcement learning and control algorithms

Symbol	Description
$V^\star(s)$	The optimal state-value function, representing the maximum expected return at state s.
$\pi(s,a)$	The policy probability of selecting action a at state s.
$P^a_{ss'}$	The transition probability from state s to state s' by action a.
$R^a_{ss'}$	The reward for transitioning from state s to state s' under action a.
$Q^\star(s,a)$	The optimal action-value function, representing the maximum expected return for taking action a at state s.
$\mathcal{H}(\pi(\cdot\|s_t))$	The policy entropy, representing the randomness of the policy at state s_t.

References

1. Abdelmaksoud, S.I.; Al-Mola, M.H.; Abro, G.E.M.; Asirvadam, V.S. In-Depth Review of Advanced Control Strategies and Cutting-Edge Trends in Robot Manipulators: Analyzing the Latest Developments and Techniques. *IEEE Access* **2024**, *12*, 47672–47701. [CrossRef]
2. Poór, P.; Broum, T.; Basl, J. Role of collaborative robots in industry 4.0 with target on education in industrial engineering.In Proceedings of the 2019 4th International Conference on Control, Robotics and Cybernetics (CRC), Tokyo, Japan, 27–30 September 2019; pp. 42–46.
3. Hu, Y.; Wang, W.; Liu, H.; Liu, L. Reinforcement learning tracking control for robotic manipulator with kernel-based dynamic model. *IEEE Trans. Neural Netw. Learn. Syst.* **2020**, *31*, 3570–3578. [CrossRef] [PubMed]
4. Chotikunnan, P.; Chotikunnan, R. Dual design pid controller for robotic manipulator application. *J. Robot. Control (JRC)* **2023**, *4*, 23–34. [CrossRef]
5. Dou, W.; Ding, S.; Yu, X. Event-triggered second-order sliding-mode control of uncertain nonlinear systems. *IEEE Trans. Syst. Man Cybern. Syst.* **2023**, *53*, 7269–7279. [CrossRef]
6. Pan, J. Fractional-order sliding mode control of manipulator combined with disturbance and state observer.*Robot. Auton. Syst.* **2025**, *183*, 104840. [CrossRef]
7. Li, T.; Li, S.; Sun, H.; Lv, D. The fixed-time observer-based adaptive tracking control for aerial flexible-joint robot with input saturation and output constraint. *Drones* **2023**, *7*, 348. [CrossRef]
8. Cho, M.; Lee, Y.; Kim, K.S. Model predictive control of autonomous vehicles with integrated barriers using occupancy grid maps. *IEEE Robot. Autom. Lett.* **2023**, *8*, 2006–2013. [CrossRef]
9. Deng, W.; Zhou, H.; Zhou, J.; Yao, J. Neural network-based adaptive asymptotic prescribed performance tracking control of hydraulic manipulators.*IEEE Trans. Syst. Man Cybern. Syst.* **2022**, *53*, 285–295. [CrossRef]
10. Li, S.; Nguyen, H.T.; Cheah, C.C. A theoretical framework for end-to-end learning of deep neural networks with applications to robotics. *IEEE Access* **2023**, *11*, 21992–22006. [CrossRef]
11. Zhu, Z.; Lin, K.; Jain, A.K.; Zhou, J. Transfer learning in deep reinforcement learning: A survey. *IEEE Trans. Pattern Anal. Mach. Intell.* **2023**, *45*, 13344–13362. [CrossRef]
12. Tran, V.P.; Mabrok, M.A.; Anavatti, S.G.; Garratt, M.A.; Petersen, I.R. Robust fuzzy q-learning-based strictly negative imaginary tracking controllers for the uncertain quadrotor systems. *IEEE Trans. Cybern.* **2023**, *53*, 5108–5120. [CrossRef] [PubMed]
13. Liu, J.; Zhou, Y.; Gao, J.; Yan, W. Visual servoing gain tuning by sarsa: An application with a manipulator.In Proceedings of the 2023 3rd International Conference on Robotics and Control Engineering, Nanjing, China, 12–14 May 2023; pp.103–107.
14. Xu, H.; Fan, J.; Wang, Q. Model-based reinforcement learning for trajectory tracking of musculoskeletal robots. In Proceedings of the 2023 IEEE International Instrumentation and Measurement Technology Conference (I2MTC), Kuala Lumpur, Malaysia, 22–25 May 2023; pp. 1–6.
15. Li, X.; Shang, W.; Cong, S. Offline reinforcement learning of robotic control using deep kinematics and dynamics. *IEEE/ASME Trans. Mechatron.* **2023**, *29*, 2428–2439. [CrossRef]
16. Zhang, S.; Pang, Y.; Hu, G. Trajectory-tracking control of robotic system via proximal policy optimization. In Proceedings of the 2019 IEEE International Conference on Cybernetics and Intelligent Systems (CIS) and IEEE Conference on Robotics, Automation and Mechatronics (RAM), Bangkok, Thailand, 18–20 November 2019; pp.380–385.
17. Schulman, J.; Wolski, F.; Dhariwal, P.; Radford, A.; Klimov, O. Proximal policy optimization algorithms. *CoRR arXiv* **2017**, arXiv:abs/1707.06347 .
18. Hu, Y.; Si, B. A reinforcement learning neural network for robotic manipulator control. *Neural Comput.* **2018**, *30*, 1983–2004. [CrossRef]
19. Lei, W.; Sun, G. End-to-end active non-cooperative target tracking of free-floating space manipulators. *Trans. Inst. Meas. Control* **2023**, *416*, 379–394. [CrossRef]
20. Song, D.; Gan, W.; Yao, P. Search and tracking strategy of autonomous surface underwater vehicle in oceanic eddies based on deep reinforcement learning. *Appl. Soft Comput.* **2023**, *132*, 109902. [CrossRef]
21. Ho, J.; Ermon, S. Generative adversarial imitation learning. *Adv. Neural Inf. Process. Syst.* **2016**, *29*, 2016.
22. Ning, G.; Liang, H.; Zhang, X.; Liao, H. Inverse-reinforcement-learning-based robotic ultrasound active compliance control in uncertain environments. *IEEE Trans. Ind. Electron.* **2024**, *71*, 1686–1696. [CrossRef]
23. Goodfellow, I.; Pouget-Abadie, J.; Mirza, M.; Xu, B.; Warde-Farley, D.; Ozair, S.; Courville, A.; Bengio, Y. Generative adversarial networks. *Commun. Acm* **2020**, *63*, 139–144. [CrossRef]
24. Jiang, D.; Huang, J.; Fang, Z.; Cheng, C.; Sha, Q.; He, B.; Li, G. Generative adversarial interactive imitation learning for path following of autonomous underwater vehicle. *Ocean. Eng.* **2022**, *260*, 111971. [CrossRef]
25. Chaysri, P.; Spatharis, C.; Blekas, K.; Vlachos, K. Unmanned surface vehicle navigation through generative adversarial imitation learning. *Ocean. Eng.* **2023**, *282*, 114989. [CrossRef]
26. Schulman, J.; Levine, S.; Abbeel, P.; Jordan, M.; Moritz, P. Trust region policy optimization. In Proceedings of the International Conference on Machine Learning, PMLR, Lille, France, 7–9 July 2015; pp.1889–1897.
27. Pecioski, D.; Gavriloski, V.; Domazetovska, S.; Ignjatovska, A. An overview of reinforcement learning techniques. In Proceedings of the 2023 12th Mediterranean Conference on Embedded Computing (MECO), Budva, Montenegro, 6–10 June 2023; IEEE: New York, NY, USA, 2023; pp.1–4.

28. Zhou, Y.; Lu, M.; Liu, X.; Che, Z.; Xu, Z.; Tang, J.; Zhang, Y.; Peng, Y.; Peng, Y. Distributional generative adversarial imitation learning with reproducing kernel generalization. *Neural Netw.* **2023**, *165*, 43–59. [CrossRef] [PubMed]
29. Spong, M.W.; Hutchinson, S.; Vidyasagar, M. *Robot Modeling and Control*; John Wiley & Sons: Hoboken, NJ, USA, 2020.
30. Wan, L.; Pan, Y.J.; Shen, H. Improving synchronization performance of multiple euler–lagrange systems using nonsingular terminal sliding mode control with fuzzy logic. *IEEE/ASME Trans. Mechatron.* **2021**, *27*, 2312–2321. [CrossRef]
31. Ma, Z.; Liu, Z.; Huang, P. Fractional-order control for uncertain teleoperated cyber-physical system with actuator fault. *IEEE/ASME Trans. Mechatron.* **2020**, *26*, 2472–2482. [CrossRef]
32. Forbrigger, S. Prediction-based haptic interfaces to improve transparency for complex virtual environments. 2017 .
33. Liu, Y.C.; Khong, M.H. Adaptive control for nonlinear teleoperators with uncertain kinematics and dynamics. *IEEE/ASME Trans. Mechatron.* **2015**, *20*, 2550–2562. [CrossRef]
34. Maheshwari, A.; Rautela, A.; Rayguru, M.M.; Valluru, S.K. Adaptive-optimal control for reconfigurable robots. In Proceedings of the 2023 International Conference on Device Intelligence, Computing and Communication Technologies, (DICCT), Dehradun, India, 17–18 March 2023; pp.511–514.
35. Li, F.; Fu, M.; Chen, W.; Zhang, F.; Zhang, H.; Qu, H.; Yi, Z. Improving exploration in actor–critic with weakly pessimistic value estimation and optimistic policy optimization. *IEEE Trans. Neural Netw. Learn. Syst.* **2022**, *35*, 8783–8796. [CrossRef]
36. Hochreiter, S.; Schmidhuber, J. Long short-term memory. *Neural Comput.* **1997**, *9*, 1735–1780. [CrossRef]
37. Zhang, L.; Liu, Q.; Huang, Z.; Wu, L. Learning unbiased rewards with mutual information in adversarial imitation learning. In Proceedings of the ICASSP 2023—2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), Seoul, Republic of Korea, 4–10 June 2023; pp. 1–5.
38. Huang, F.; Xu, J.; Wu, D.; Cui, Y.; Yan, Z.; Xing, W.; Zhang, X. A general motion controller based on deep reinforcement learning for an autonomous underwater vehicle with unknown disturbances. *Eng. Appl. Artif. Intell.* **2023**, *117*, 105589. [CrossRef]
39. Wang, T.; Wang, F.; Xie, Z.; Qin, F. Curiosity model policy optimization for robotic manipulator tracking control with input saturation in uncertain environment. *Front. Neurorobot.* **2024**, *18*, 1376215. [CrossRef]

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.